

Использование языка программирования Julia для решения математических задач

Часть II

Авторы:

Чуракова Юлия Романовна

*Студентка факультета математики и компьютерных наук
Кубанского Государственного Университета*

Алексеев Евгений Ростиславович

*Кандидат технических наук, доцент
Доцент кафедры информационных технологий
Кубанского Государственного Университета*

Полная версия пособия доступна в репозитории:

<https://github.com/JuliaChurakova/Julia>

Литература

1. Алексеев, Е. Р. Scilab: Решение инженерных и математических задач: учеб. издание. / Е.Р. Алексеев, К. В. Дога, О. В. Чеснокова М. – Базальт СПО; ДМК Пресс, 2024. – 440 с.
2. Балакин, А. А. Численные методы и математическое моделирование: Учебное пособие / А.А. Балакин. – Долгопрудный: Издательский Дом «Интеллект», 2022. – 288 с.
3. Деза, Е. И. Введение в криптографию: Теоретико-числовые основы защиты информации. Учебное пособие / Е. И. Деза, Л. В. Котова. – Москва: ЛЕНАНД, 2022. – 376 с.
4. Кнауб, Л. В. Теоретико-численные методы в криптографии / Л. В. Кнауб, Е. А. Новиков, Ю. А. Шитов; Сибирский федеральный университет – Красноярск: 2011. – 160 с.

Глава 15 Элементарная теория чисел

Теория чисел – это раздел математики, занимающийся изучением свойств целых чисел, их структур и взаимосвязей. Она охватывает такие вопросы, как делимость чисел, разложение на простые множители, алгоритмы нахождения наибольшего общего делителя и наименьшего общего кратного, а также другие теоретико-числовые задачи.

15.1 Теорема деления

Теорема деления (также известная как **формула деления с остатком**)

Теорема утверждает, что для любых двух натуральных чисел a и b ($a \geq b$) существует уникальная пара неотрицательных целых чисел q и r , которые удовлетворяют следующему равенству:

$$a = bq + r,$$

где:

- q – это **целая часть** от деления a на b ;
- r – это **остаток** от деления a на b ;
- $0 \leq r < b$ – остаток должен быть неотрицательным и строго меньше делителя b .

По другому это можно записать через операцию **mod**.

Операция **mod** – это операция, которая возвращает остаток от деления одного числа на другое. Операция **mod** обозначается как $a \bmod b$, где a – делимое, b – делитель, а r – это остаток от деления числа a на b .

$$a \bmod b = r,$$

15.2 Наибольший общий делитель (НОД)

В теории чисел важной задачей является нахождение **наибольшего общего делителя** (НОД) двух чисел. НОД двух чисел – это наибольшее число, которое делит оба исходных числа без остатка.

15.2.1 Алгоритм Евклида

Алгоритм Евклида позволяет находить наибольший общий делитель (НОД) двух чисел a и b с помощью последовательных делений с остатком.

Шаги алгоритма:

1. пусть a и b – два числа, $a \geq b > 0$;
2. на каждом шаге выполняем деление с остатком $a = bq + r$;
3. если $r = 0$, то НОД равен b ;
4. иначе, заменяем a на b , b на r , и повторяем шаги.

```
In [21]: function euclid(a, b)
          while b != 0
              a, b = b, a % b # одновременное обновление a и b
          end
          return a
      end

      println("Введите первое число (a)")
      a = parse{Int, readline()}
      println("a = ", a)

      println("Введите второе число (b)")
      b = parse{Int, readline()}
      println("b = ", b)

      println("Наибольший общий делитель (НОД) чисел ", a, " и ", b, " равен: ", e
```

Введите первое число (a)

a = 45

Введите второе число (b)

b = 65

Наибольший общий делитель (НОД) чисел 45 и 65 равен: 5

15.2.2 Расширенный алгоритм Евклида

Алгоритм Евклида, который мы рассмотрели ранее, позволяет находить **наибольший общий делитель (НОД)** двух чисел. Однако, в некоторых задачах нам нужно не только вычислить НОД, но и найти коэффициенты, которые позволяют выразить НОД как линейную комбинацию этих чисел. Именно для этого используется **расширенный алгоритм Евклида**.

Пусть даны два целых числа a и b . Необходимо найти такие целые числа x и y , что:

$$\text{НОД}(a, b) = a \cdot x + b \cdot y,$$

где x и y – целые коэффициенты, называемые **коэффициентами линейной комбинации** для чисел a и b .

Расширенный алгоритм Евклида строится на основе стандартного алгоритма Евклида, но дополнительно отслеживаются коэффициенты x и y .

Мы начинаем с инициализации этих коэффициентов: $x_0 = 1, y_0 = 0$ для a и $x_1 = 0, y_1 = 1$ для b .

Далее, на каждом шаге, помимо вычисления нового остатка, также вычисляются новые коэффициенты x_i и y_i .

$$x_{i+1} = x_{i-1} - x_i \cdot q_{i-1}$$

$$y_{i+1} = y_{i-1} - y_i \cdot q_{i-1}$$

Когда остаток становится равным нулю, последний ненулевой остаток будет являться НОД. Коэффициенты x и y в этот момент будут такими, что они удовлетворяют уравнению:

$$\text{НОД}(a, b) = a \cdot x + b \cdot y.$$

```
In [22]: function extended_euclid(a, b)
    x0, x1 = 1, 0
    y0, y1 = 0, 1

    while b != 0
        q, r = a ÷ b, a % b
        a, b = b, r
        x0, x1 = x1, x0 - q * x1
        y0, y1 = y1, y0 - q * y1
    end

    return a, x0, y0
end

println("Введите первое число (a)")
a = parse{Int, readline()}
println("a = ", a)
```

```
println("Введите второе число (b)")
b = parse{Int, readline()}
println("b = ", b)

gcd, x, y = extended_euclid(a, b)
println("Наибольший общий делитель (НОД) чисел ", a, " и ", b, " равен: ", gcd)
println("Коэффициенты: x = ", x, ", y = ", y)
```

Введите первое число (a)

a = 345495825

Введите второе число (b)

b = 345945

Наибольший общий делитель (НОД) чисел 345495825 и 345945 равен: 15

Коэффициенты: x = -1250, y = 1248377

В Julia есть стандартная функция **gcdx()** из встроенного модуля:

```
In [34]: a = 345495825
         b = 345945
         gcdx(a, b) # Возвращает кортеж (НОД, x, y), где x*a + y*b = НОД
```

```
Out[34]: (15, -1250, 1248377)
```

15.3 Простые и составные числа

В теории чисел различают два основных класса чисел — простые и составные.

Простое число – это натуральное число больше единицы, которое делится только на 1 и на себя. То есть, если p – простое число, то его единственными делителями являются 1 и p .

Составное число – это натуральное число больше единицы, которое не является простым, то есть оно имеет больше двух делителей. Составные числа можно разложить на произведение простых чисел.

15.4.1 Метод пробного деления

Один из самых прямолинейных методов проверки, является **метод пробного деления**. В этом методе мы проверяем, делится ли число n на любое число от 2 до $\sqrt{n}$. Если n делится на одно из таких чисел, оно составное, если нет – число простое.

Почему достаточно проверять делители до $\sqrt{n}$? Это можно объяснить следующим образом:

пусть n – составное число, то есть оно может быть представлено как произведение двух чисел a и b :

$$n = a \times b$$

Если оба множителя a и b больше $\sqrt{n}$, то их произведение будет больше n , что невозможно, потому что $a \times b = n$. Поэтому хотя бы один из множителей должен быть меньше или равен $\sqrt{n}$. Следовательно, если мы проверяем делители числа n до $\sqrt{n}$, мы гарантированно находим хотя бы один из множителей, если n составное.

```
In [10]: function is_prime(n)
    if n < 2
        return false # Числа < 2 не являются простыми
    end
    for i in 2:isqrt(n) # Проверяем делители от 2 до √n
        if n % i == 0 # Если n делится на i, то оно составное
            return false
        end
    end
    return true # Если делителей не найдено, число простое
end

println("Введите число")
n = parse{Int, readline()}
println("Введенное значение: ", n)

if is_prime(n)
    println("Число $n простое.")
else
    println("Число $n составное.")
end
```

```
Введите число
Введенное значение: 11
Число 11 простое.
```

Хотя метод пробного деления прост и понятен, он имеет значительные недостатки, особенно для больших чисел. Время работы резко увеличивается с ростом числа n , для очень больших чисел этот метод становится слишком медленным и непрактичным.

15.4.2 Решето Эратосфена

Этот алгоритм позволяет найти все простые числа до заданного числа N . Он состоит в поочередном вычеркивании составных чисел, начиная с 2. После завершения алгоритма все оставшиеся числа – простые.

Алгоритм можно описать следующим образом:

1. запишем все целые числа от 2 до N . $S = 2, 3, 4, \dots, N$ – исходный список чисел;
2. пусть переменная p изначально равно 2 – первому простому числу, вычеркиваем из списка все числа от $2 \cdot p$ до N , считая шагами по p ;

3. находим первое незачёркнутое число в списке, большее p , и присваиваем это число переменной p ;
4. повторяем шаги 2 и 3, пока значение $p^2 \leq N$).

```
In [11]: function Eratosthenes(n)
    # Создаём массив для простых чисел
    prime = fill(true, n) # Все числа считаем простыми (true)
    prime[1] = false # 1 не является простым числом

    #Для каждого числа p от 2 до √n
    for p in 2:isqrt(n)
        if prime[p] == true
            # Вычеркиваем все кратные j, начиная с j^2
            for k in p^2:p:n
                prime[k] = false
            end
        end
    end

    #Выводим все простые числа
    for p in 2:n
        if prime[p]
            println(p)
        end
    end
end

println("Введите n")
n = parse{Int, readline()}
println("n = ", n)
Eratosthenes(n)
```

```
Введите n
n = 20
2
3
5
7
11
13
17
19
```

15.4.3 Тест Ферма

Этот тест основан на **малой теореме Ферма**, которая утверждает, что для любого простого числа p и для любого числа a , которое не делится на p , выполняется следующее условие:

$$a^{p-1} \equiv 1 \pmod{p}$$

Это значит, что если число n простое, то для любого числа a где $1 < a < n - 1$ будет выполняться $a^{n-1} \equiv 1 \pmod{n}$. Если это условие не

выполняется для некоторого числа a , то n составное.

```
In [12]: # Функция для быстрого возведения в степень по модулю
function modd(base, exp, mod)
    result = BigInt(1) # Устанавливаем результат равным 1
    base = base % mod # Вычисляем основание по модулю
    while exp > 0 # Цикл, пока показатель степени больше 0
        if exp % 2 == 1 # Если степень нечетная
            result = (result * base) % mod # Умножаем результат на основание
        end
        exp ÷= 2 # Делим степень на 2
        base = (base * base) % mod # Возводим основание в квадрат и находим
    end
    return result
end

# Функция для теста Ферма
function fermat_test(n, iterations=25)
    # Проверка на базовые случаи
    if n < 2
        return false # Числа < 2 не простые
    end
    if n == 2
        return true # 2 – простое число
    end
    if n % 2 == 0
        return false # Чётные числа больше 2 не простые
    end

    # Выполнение теста Ферма для случайных оснований
    for i in 1:iterations
        a = rand(2:n-2) # Генерируем случайное a, где 1 < a < n-1
        if modd(a, n-1, n) != 1
            return false # Если a^(n-1) mod n != 1, то n составное
        end
    end

    return true # Если тест прошёл все итерации, то число вероятно простое
end

# Ввод числа и проверка на простоту
println("Введите число")
n = parse{Int, readline()} # Читаем число с консоли

if fermat_test(n)
    println("Введенное число $n простое.")
else
    println("Введенное число $n составное.")
end
```

Введите число

Введенное число 233 простое.

15.4.4 Тест Миллера-Рабина

Тест Миллера-Рабина – это вероятностный тест, используемый для проверки простоты чисел. Он является псевдопростым тестом, что означает, что он может ошибаться, но вероятность ошибки можно уменьшить путём многократного применения теста.

Основная идея алгоритма:

1. алгоритм проверки простоты числа начинается с базовых случаев: если $n \leq 1$ — не простое, если $n = 2$ или 3 – простое, если $n > 2$ и чётное – составное;
2. далее представляем число $n - 1$ как $n - 1 = 2^s \cdot d$, где d нечётно;
3. выбираем случайное число a , где $2 \leq a \leq n - 2$, и проверяем условие $a^d \bmod n$. Если результат равен 1 или $n - 1$, то n с высокой вероятностью простое;
4. если условие не выполнено, то вычисляем $a^{2^i \cdot d} \bmod n$ для $i = 1, 2, \dots, r - 1$. Если на каком-то шаге i результат равен $n - 1$, то число вероятно простое. Если результат снова становится равным 1, то число n составное;
5. если число n не проходит проверку для выбранного числа a , оно составное. Повторяя тест с разными основаниями a , можно с высокой вероятностью утверждать, что число простое, если оно прошло все проверки.

```
In [13]: # Функция для быстрого возведения в степень по модулю
function modd(base, exp, mod)
    result = BigInt(1) # Устанавливаем результат равным 1
    base = base % mod # Основание по модулю
    while exp > 0 # Цикл, пока степень больше 0
        if exp % 2 == 1 # Если степень нечётная
            result = (result * base) % mod # Умножаем на основание и берём
        end
        exp ÷= 2 # Делим степень на 2
        base = (base * base) % mod # Возводим основание в квадрат по модулю
    end
    return result
end

function is_prime(n)
    iterations = 25 # Количество итераций для теста Миллера-Рабина

    # Проверка на базовые случаи
    if n < 2
        return false # Числа < 2 не простые
    end
    if n == 2
        return true # 2 – простое число
```

```

end
if n % 2 == 0
    return false # Чётные числа больше 2 не простые
end

# Разложение  $n - 1 = 2^r * d$ , где  $d$  нечётно
d = n - 1
r = BigInt(0)
while d % 2 == 0 # Пока  $d$  чётное, делим его на 2
    d ÷= 2
    r += 1
end

# Выполнение теста Миллера-Рабина для случайных оснований
for i in 1:iterations
    a = rand(2:n-2) # Генерируем случайное  $a$ , где  $2 \leq a \leq n-2$ 
    x = modd(a, d, n) # Вычисляем  $x = a^d \bmod n$ 

    # Если  $x$  равно 1 или  $n-1$ , продолжаем
    if x != 1 && x != n - 1
        composite = true # Предполагаем, что  $n$  составное
        temp_r = r
        while temp_r > 0 # Цикл для проверки степеней  $x$ 
            x = modd(x, 2, n) #  $x = x^2 \bmod n$ 
            if x == n - 1
                composite = false #  $n$  простое, выходим
                break
            end
            temp_r -= 1
        end
    end

    # Если  $x$  не стало равно  $n-1$ , число составное
    if composite
        return false # Число составное
    end
end

return true # Если тест прошёл все итерации, число вероятно простое
end

# Ввод числа и проверка на простоту
println("Введите число:")
n = parse(BigInt, readline())

if is_prime(n)
    println("Введенное число  $n$  простое.")
else
    println("Введенное число  $n$  составное.")
end
end

```

Введите число:

Введенное число 233 простое.

Тест Миллера-Рабина особенно полезен для больших чисел, он быстро определяет, является ли число простым. Это значительно ускоряет процесс

проверки, делая алгоритм подходящим для чисел, используемых в криптографии и других областях, где важна скорость обработки данных. Несмотря на то, что тест является вероятностным, его использование с достаточным количеством итераций даёт очень высокую вероятность правильного результата, что делает его удобным для практического применения.

15.4 Разложение чисел на множители

Каждое целое число $n > 1$ можно разложить на произведение простых чисел, то есть существует представление:

$$n = p_1^{a_1} p_2^{a_2} \cdots p_k^{a_k},$$

где $p_1, p_2, \dots, p_k$ – простые числа, а $a_1, a_2, \dots, a_k$ – неотрицательные целые числа.

Алгоритм разложения на множители методом пробных делений.

Метод пробных делений (или метод деления с остатком) является одним из самых простых и интуитивно понятных способов разложения числа на простые множители. Суть метода заключается в том, чтобы попытаться разделить число n на все возможные простые числа, начиная с самого маленького, пока не будут найдены все простые множители числа.

Этот метод работает путем последовательного деления числа на простые числа, начиная с 2 и продолжая до тех пор, пока число не станет равным 1. Если при делении на какое-то число остаток от деления равен нулю, то это число является делителем исходного числа, и его нужно записать как множитель.

```
In [15]: function factorize(n)
    factors = Int[] # Массив для хранения множителей
    i = 2 # Начинаем с самого маленького простого числа

    while n > 1
        while n % i == 0 # Если число делится на i
            push!(factors, i) # Добавляем i в список множителей
            n ÷= i # Делим n на i
        end
        i += 1 # Увеличиваем i и продолжаем искать следующие делители
    end
    return factors
end

println("Введите n")
n = parse{Int, readline()})
```

```
println("n = ", n)
println("Множители числа $n: ", factorize(n))
```

Введите n

n = 123456789

Множители числа 123456789: [3, 3, 3607, 3803]

15.5 Функция Эйлера

Функция Эйлера $\varphi(n)$ для целого числа n определяет количество чисел, которые взаимно просты с n , то есть таких чисел, которые не имеют общих делителей с n , кроме 1.

Свойства функции Эйлера:

1. **если n простое**, то $\varphi(n) = n - 1$;
2. **если n составное и разлагается на простые множители**, то функция Эйлера для числа n , разлагаемого на простые множители $p_1, p_2, \dots, p_k$, вычисляется по формуле:

$$\varphi(n) = n \cdot \left(1 - \frac{1}{p_1}\right) \cdot \left(1 - \frac{1}{p_2}\right) \cdot \dots \cdot \left(1 - \frac{1}{p_k}\right)$$

```
In [18]: function euler(n)
    result = n # Начальное значение для результата

    # Перебор простых чисел от 2 до sqrt(n) с шагом 1
    for p in 2:isqrt(n)
        if n % p == 0 # Если p делит n
            # Уменьшаем результат с учетом p
            while n % p == 0
                n ÷= p # Делим n на p, пока p делит n
            end
            result -= result ÷ p # Применяем формулу (1 - 1/p)
        end
    end

    # Если n > 1, то n – это последний простой множитель
    if n > 1
        result -= result ÷ n # Применяем формулу (1 - 1/n) для последнего p
    end

    return result
end

n = 30
println("Функция Эйлера для числа $n: ", euler(n))
```

Функция Эйлера для числа 30: 8

15.6 Библиотека Primes

Библиотека **Primes** – для работы с простыми числами и выполнения задач теории чисел.

factor(n) – возвращает разложение числа на простые множители в виде словаря, где ключи – простые делители, а значения – их кратности.

```
In [5]: using Primes  
println(factor(1000000000000000000000000))
```

```
Primes.Factorization{Int128}(173 => 1, 3739 => 1, 336958757 => 1, 4587981777
1 => 1)
```

isprime(n) – проверяет, является ли число простым.

```
In [46]: isprime(17)
```

```
Out[46]: true
```

primes(start, stop) – генерирует список простых чисел в заданном диапазоне.

```
In [47]: primes(10, 20)
```

```
Out[47]: 4-element Vector{Int64}:
          11
          13
          17
          19
```

totient(n) – вычисляет функцию Эйлера.

```
In [50]: totient(30)
```

```
Out[50]: 8
```

Для получения дополнительной информации о пакете см. [документацию](#).